

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №1
по дисциплине «Методы оптимизации»
Тема: «Решение задач линейного программирования»

Выполнила:
Степанова Екатерина Андреевна, К3341
поток 1.2

Санкт-Петербург
2026 г.

Введение

Практическое задание по теме «Задачи линейного программирования»

В рамках данной работы вы закрепите навыки работы с математической постановкой задачи линейного программирования, получения из нее допустимого канонического вида и ее решения с помощью симплекс-метода.

Что дано на входе?

Общая задача линейного программирования – математическая запись: целевая функций (он же критерий эффективности); ограничения.

Что необходимо сделать или шаги решения?

1. Привести задачу к каноническому виду.
2. Сформулировать вспомогательную задачу.
3. Решить вспомогательную задачу.
4. Перейти к основной задаче (если есть возможность).
5. Решить основную задачу.
6. Записать ответ в форме: оптимальная точка , значения целевой функции в оптимальной точке: .
7. Решить задачу в Excel (используйте надстройку «Solver») и сравнить ответы.

Шаги решения выполняются вручную и оформляются либо шариковой или иной ручкой на бумаге (допускается фотографировать соответствующие листы для последующего внесения изображений в документ, содержание на листах должно быть разборчивым), либо в документе с помощью средств ввода текстов и формул.

Ход решения

Вариант 4

Минимизировать $Z = 3x_1 + x_2 + 4x_3 + 2x_4$
при условиях:

$$2x_1 + x_2 + x_3 \leq 8$$

$$x_1 + x_3 + x_4 = 5$$

$$x_2 + x_4 \geq 4$$

Реализация ручного метода

Вариант 4. Степанова Е.А

Задача: Минимизировать $Z = 3x_1 + x_2 + 4x_3 + 2x_4$

при условиях:

$$\begin{cases} 2x_1 + x_2 + x_3 \leq 8 \\ x_1 + x_3 + x_4 = 5 \\ x_2 + x_4 \geq 4 \\ x_1, x_2, x_3, x_4 \geq 0 \end{cases} \Rightarrow \begin{cases} 2x_1 + x_2 + x_3 + 0 \cdot x_4 \leq 8 \\ x_1 + 0 \cdot x_2 + x_3 + x_4 = 5 \\ 0 \cdot x_1 + x_2 + 0 \cdot x_3 + x_4 \geq 4 \\ x_1, \dots, x_4 \geq 0 \end{cases}$$

Решение.

1) Проверим задачу в каноническом виде.

$$W(x) = 3x_1 + x_2 + 4x_3 + 2x_4 \rightarrow \min$$

$$\begin{cases} 2x_1 + x_2 + x_3 + \textcircled{+}x_5 = 8 \quad (+, \text{т.к. } \leq) & (1) \\ x_1 + 0 \cdot x_2 + x_3 + x_4 = 5 & (2) \\ 0 \cdot x_1 + x_2 + 0 \cdot x_3 + x_4 + \textcircled{-}x_6 = 4 \quad (-, \text{т.к. } \geq) & (3) \\ x_1, \dots, x_6 \geq 0. \end{cases}$$

2) Сформулируем вспомогательную задачу.

Введем свой базис путем добавления фиктивных переменных в уравнения.

В (1) уравнении уже есть переменная с коэффициентом +1 (x_5). Поэтому добавим фиктивные переменные x_7 и x_8 в уравнения (2) и (3) соответственно.

$$\begin{cases} 2x_1 + x_2 + x_3 + 0 \cdot x_4 + \textcircled{+}x_5 = 8 \\ x_1 + 0 \cdot x_2 + x_3 + x_4 + 0 \cdot x_5 + \textcircled{+}x_7 = 5 \\ x_2 + x_4 - x_6 + \textcircled{+}x_8 = 4 \\ x_1, x_2, \dots, x_8 \geq 0 \end{cases}$$

$$W'(x') = x_7 + x_8 \rightarrow \min$$

(1)

3) Решим вспомог. задачу.

Выразим $W'(x')$ через свободные переменные.

$$x_7 = 5 - x_1 - x_3 - x_4$$

$$x_8 = 4 - x_2 - x_4 + x_6$$

$$W'(x') = x_7 + x_8 = (5 - x_1 - x_3 - x_4) + (4 - x_2 - x_4 + x_6) = \\ = \textcircled{9} - x_1 - x_2 - x_3 - 2x_4 + x_6 \rightarrow \min$$

Построим СТ:

	1	2	3	4	6	
5	2	1	1	0	0	8
7	1	0	1	1	0	5
8	0	1	0	1	-1	4
	-1	-1	-1	-2	1	-9

1) Ст. Разреш. столбец $\forall c_j < 0$

$$j = (1, 4)$$

$$\textcircled{j^* = 4}$$

2) Ст. Разреш. строка

$$\min \frac{b_i}{a_{ij^*}}$$

$$\textcircled{i^* = 8}$$

т.к. $\frac{4}{1} = 4 \rightarrow \min.$ значение

Итого разреш. элемент $a_{i^*j^*} = 1$

3) Замена базисной и свободной переменных.

	1	2	3	8	6	
5	2	1	1	0	0	8
7	1	-1	1	-1	1	1
4	0	1	0	1	-1	4
	-1	1	-1	2	-1	-1

4) Ст 4. Перерасчет разр. элемента

$$\hat{a}_{i^*j^*} = \frac{1}{a_{i^*j^*}} = \frac{1}{1} = 1$$

5) Ст 5. Перерасчет разреш. строки

$$\hat{a}_{ij} = \frac{a_{ij}}{a_{i^*j^*}} \quad j = 1, 6 \quad j \neq j^*$$

т.к. у нас $a_{i^*j^*} = 1$, то в строке элементы ост. неизменны.

6) Ст 6. Перерасчет разр. столбца

$$\hat{a}_{ij^*} = -\frac{a_{ij^*}}{a_{i^*j^*}} \quad \hat{c}_{j^*} = -\frac{c_{j^*}}{a_{i^*j^*}}$$

В тех местах, где в разреш. столбце или строке 0, элемент, перпендикулярный такой строке или столбцу, равен своему прошлому значению.

7) Ст 7. Перерасчет оставшихся элементов.

$$a_{72} = 0 - \frac{1 \cdot 1}{1} = -1$$

$$a_{76} = 0 - \frac{(-1) \cdot 1}{1} = 1$$

$$b_7 = 5 - \frac{4 \cdot 1}{1} = 1$$

$$c_2 = -1 - \frac{1 \cdot (-2)}{1} = -1 + 2 = 1$$

$$\hat{c}_6 = 1 - \frac{(-1) \cdot (-2)}{1} = 1 - 2 = -1$$

$$\hat{Q} = -9 - \frac{4 \cdot (-2)}{1} = -9 + 8 = -1$$

(2)

Аналогичным образом выбираем разр. столбец и строку 4 способа зрелищности ст. $C_{1j^*} = 1$; т.к. у нас есть отрицательное наименьшее значение (-1) , то мы можем выбрать любой столбец с $C = -1$.

Ст2 $j^* = 4$

Ст3. Замена базисной и свободной переменной

	7	2	3	8	6	
5	-2	3	-1	2	-2	6
1	1	-1	1	-1	1	1
4	0	1	0	1	-1	4
	1	0	0	1	0	0

Ст4. Разрешающий элемент $a_{1j^*} = \frac{1}{1} = 1$

Ст5. Перерасчет разр. строки

$$\hat{a}_{ij^*} = \frac{a_{ij^*}}{a_{1j^*}} \quad j \neq j^*$$

т.к. делим на 1 ($a_{1j^*} = 1$); то просто дублируем значение строки из прошлой таблицы.

Ст6 Перерасчет разреш. столбца $\hat{a}_{ij^*} = -\frac{a_{ij^*}}{a_{1j^*}} \quad i \neq j^* \quad \hat{c}_{j^*} = -\frac{c_{j^*}}{a_{1j^*}}$

Меняем знак и сохраняем значение из прошлой таблицы

Ст7 Перерасчет ост. элементов

$$\hat{a}_{52} = 1 - \frac{2 \cdot (-1)}{1} = 3$$

$$\hat{a}_{53} = 1 - \frac{1 \cdot 2}{1} = -1$$

$$\hat{a}_{58} = 0 - \frac{(-1) \cdot 2}{1} = 2$$

$$\hat{a}_{56} = 0 - \frac{1 \cdot 2}{1} = -2$$

$$\hat{b}_5 = 8 - \frac{1 \cdot 2}{1} = 6$$

$$\hat{c}_2 = 1 - \frac{(-1) \cdot (-1)}{1} = 0$$

$$\hat{c}_3 = -1 - \frac{1 \cdot (-1)}{1} = 0$$

$$\hat{c}_8 = 2 - \frac{(-1) \cdot (-1)}{1} = 1$$

$$\hat{c}_6 = -1 - \frac{1 \cdot (-1)}{1} = 0$$

$$\hat{Q} = -1 - \frac{1 \cdot (-1)}{1} = 0$$

В последней строке СТ больше нет отриц. элементов,
 $\hat{Q} = 0 \Rightarrow$ допустимый базис найден (x_2, x_3, x_6)

(3)

$$Z = 3x_1 + x_2 + 4x_3 + 2x_4$$

Выразим ^{часть базиса} x_1 и x_4 из систем. - таблицей

$$x_1 - x_2 + x_3 + x_6 = 1 \Rightarrow x_1 = 1 + x_2 - x_3 - x_6$$

$$x_4 + x_2 - x_6 = 4 \Rightarrow x_4 = 4 - x_2 + x_6$$

Подставим в целевую функцию.

$$\begin{aligned} Z &= 3(1 + x_2 - x_3 - x_6) + x_2 + 4x_3 + 2(4 - x_2 + x_6) = \\ &= 3 + 3x_2 - 3x_3 - 3x_6 + x_2 + 4x_3 + 8 - 2x_2 + 2x_6 = \\ &= 11 + 2x_2 + x_3 - x_6 \end{aligned}$$

Строим СТ:

	2	3	6	
5	3	-1	-2	6
1	-1	1	1	1
4	1	0	-1	4
	2	1	-1	-11

$$Ст\ j^* = 6$$

$$Ст\ i^* = 1$$

Ст 3. Замена базисной и свободной переменной

	2	3	1	
5	1	1	2	8
6	-1	1	1	1
4	0	1	1	5
	1	2	1	

$$Ст\ 4\ a_{i^*j^*} = \frac{1}{a_{ij^*}} = \frac{1}{1}$$

Ст 5 размени. строка

Ст 6 размени. столбец

Ст 4 осв. элемент

$$\hat{b}_4 = 4 - \frac{1 \cdot (-1)}{1} = 5$$

$$\hat{c}_2 = 2 - \frac{(-1) \cdot (-1)}{1} = 1$$

$$\hat{c}_3 = 1 - \frac{(1) \cdot (-1)}{1} = 2$$

$$\hat{Q} = -11 - \frac{(1) \cdot (-1)}{1} = -10$$

$$\hat{a}_{52} = 3 - \frac{(-2) \cdot (-1)}{1} = 1$$

$$\hat{a}_{53} = -1 - \frac{(1) \cdot (-1)}{1} = 1$$

$$\hat{b}_5 = 6 - \frac{1 \cdot (-2)}{1} = 8$$

$$\hat{a}_{42} = 1 - \frac{(1) \cdot (-1)}{1} = 0$$

$$\hat{a}_{43} = 0 - \frac{(1) \cdot (-1)}{1} = 1$$

(4)

Основная задача решена. $-Q = -10 \Rightarrow Z = 10$

$$x_4 = 5, x_5 = 8, x_6 = 1$$

$$x_1 = 0, x_2 = 0, x_3 = 0$$

Ответ:

$$x^* = (\overset{x_1}{0}, \overset{x_2}{0}, \overset{x_3}{0}, \overset{x_4}{5})$$

$$W(x^*) = 10.$$

Реализация в Excel

Для решения этого пункта я использовала microsoft excel с надстройкой “Solver” и google sheets с расширением “OpenSolver”.

Объясню решение на примере OpenSolver.

Нам дан задача

Минимизировать $Z = 3x_1 + x_2 + 4x_3 + 2x_4$
при условиях:

$$\begin{aligned} 2x_1 + x_2 + x_3 &\leq 8 \\ x_1 + x_3 + x_4 &= 5 \\ x_2 + x_4 &\geq 4 \end{aligned}$$

1. Для начала запишем все коэффициенты целевой функции Z .

с	3	1	4	2

$$Z = \underline{3}x_1 + \underline{1}x_2 + \underline{4}x_3 + \underline{2}x_4$$

2. Запишем начальные(предполагаемые) значения x целевой функции, обычно в начале ставят все x равные 0, но можно предположить и свои значения, на расчеты это никак не повлияет, так как расширение будет потом самостоятельно пересчитывать x .

c	3	1	4	2
x	0	0	0	5

$$Z = 3x_1 + x_2 + 4x_3 + 2x_4$$

3. Выпишем коэффициенты ограничений

C1	2	1	1	0
C2	1	0	1	1
C3	0	1	0	1

4. Справа от этих коэффициентов необходимо выделить столбец под получающиеся значения, знак неравенства/равенства, требуемое условие

C1	2	1	1	0	0	<=	8
C2	1	0	1	1	5		5
C3	0	1	0	1	5	>=	4

$$\begin{array}{l} 2x_1 + x_2 + x_3 \\ x_1 + x_3 + x_4 \\ x_2 + x_4 \end{array} \begin{array}{l} \leq 8 \\ = 5 \\ \geq 4 \end{array}$$

5. В столбец, который мы выделили для расчеты текущего значения каждого ограничения при пересчете x, необходимо вставить формулу

Диапазон x1, x2, x3, x4 я отметила названием exs для удобства использования.

exs	fx 5								
	A	B	C	D	E	F	G	H	I
1									
2									
3									
4									
5									
6									
7									
8									
9									
10									
11									
12									
13		c	3	1	4	2	10		
14		x	0	0	0	5			
15									
16									
17		C1	2	1	1	0	0 <=		8
18		C2	1	0	1	1	5		5
19		C3	0	1	0	1	5 >=		4
20									
21									
22									
23									

G17	fx =СУММПРОИЗВ(exs; C17:F17)								
	A	B	C	D	E	F	G	H	I
1									
2									
3									
4									
5									
6									
7									
8									
9									
10									
11									
12									
13		c	3	1	4	2	10		
14		x	0	0	0	5			
15									
16									
17		C1	2	1	1	0	0 <=		8
18		C2	1	0	1	1	5		5
19		C3	0	1	0	1	5 >=		4
20									
21									

В ячейку мы вставляем формулу =СУММПРОИЗВ(exs; C17:F17), а затем растягиваем на все ограничения.

Что делает эта формула?

Каждый коэффициент перемножает с соответствующим хб а затем суммирует.

6. Похожую формулу необходимо реализовать и для целевой функции

Минимизировать $Z = 3x_1 + x_2 + 4x_3 + 2x_4$
при условиях:

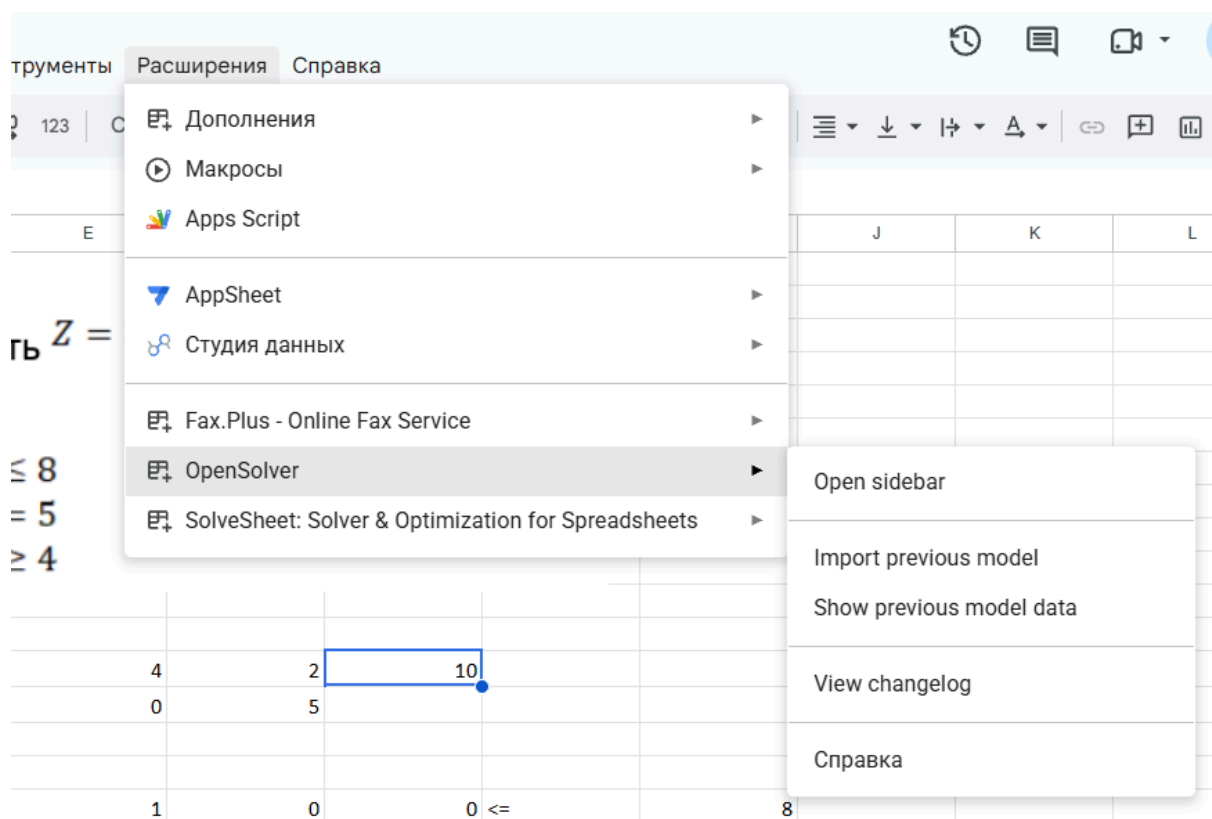
$$2x_1 + x_2 + x_3 \leq 8$$

$$x_1 + x_3 + x_4 = 5$$

$$x_2 + x_4 \geq 4$$

	с	3	1	4	2	10
	х	0	0	0	5	
	C1	2	1	1	0	0 ≤
	C2	1	0	1	1	5 =
	C3	0	1	0	1	5 ≥

7. Необходимо нажать на кнопку расширения и выбрать OpenSolver(если расширения нет - установить в магазине).



8. Открываем панель управления -> Open sidebar

OpenSolver ✕

Sheet: Лист1 ↕ Current Sheet

Objective Cell:
 'Лист1'!G13 Update Clear

Objective Sense:
☒ minimise ☐ target value:
☐ maximise

Variable Cells:
 'Лист1'!C14:F14 ▲ ▼ Add
Update Delete

☒ Unconstrained variables non-negative

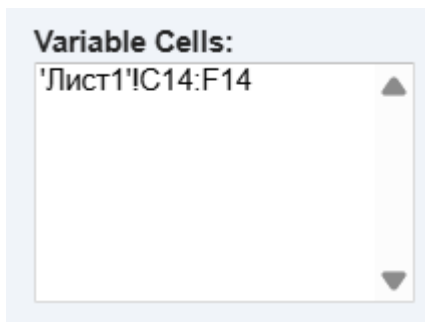
Constraints:
 <Add new constraint> ▲
 'Лист1'!G18 = 'Лист1'!I18
 'Лист1'!G17 <= 'Лист1'!I17
 'Лист1'!G19 >= 'Лист1'!I19 ▼

Selected Constraint:
 Update

9. В Objective Sense выбираем что хотим сделать, в нашем случае - минимизировать функцию

Objective Sense:
☒ minimise
☐ maximise

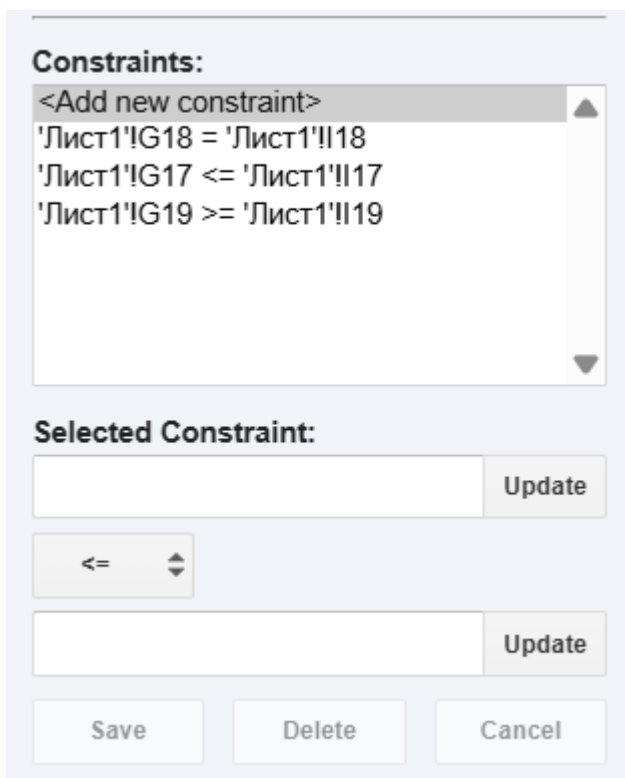
10. В Variable Cells указываем диапазон x, значения которых мы хотим подобрать. В моем случае, x хранятся в ячейках C14:F14



Variable Cells:

'Лист1!C14:F14

11. В Constraints указываем все данные ограничения



Constraints:

<Add new constraint>

'Лист1!G18 = 'Лист1!I18

'Лист1!G17 <= 'Лист1!I17

'Лист1!G19 >= 'Лист1!I19

Selected Constraint:

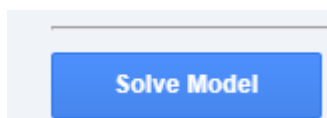
Update

<=

Update

Save Delete Cancel

12. Нажимаем на кнопку Solve Model и получаем ответ



Solve Model

Ниже представлен пример реализации с помощью Microsoft Excel.

G2

✕

✓

fx

	A	B	C	D	E	F	G	H	I	J
1										
2		c	3	1	4	2	10			
3		x	0	0	0	5				
4										
5										
6		C1	2	1	1	0	0	<=	8	
7		C2	1	0	1	1	5	=	5	
8		C3	0	1	0	1	5	>=	4	
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										
21										
22										
23										
24										
25										
26										
27										
28										
29										
30										
31										

Параметры поиска решения

Оптимизировать целевую функцию:

SG\$2

↑

До:

☐ Максимум ☒ Минимум ☐ Значения:

0

Изменяя ячейки переменных:

eks

↑

В соответствии с ограничениями:

SG\$6 <= \$I\$6
SG\$7 = \$I\$7
SG\$8 >= \$I\$8

Добавить

Изменить

Удалить

Сбросить

Загрузить/сохранить

☒ Сделать переменные без ограничений неотрицательными

Выберите метод решения:

Поиск решения лин. задач симплекс-методом

Параметры

Метод решения

Для гладких нелинейных задач используйте поиск решения нелинейных задач методом ОПГ, для линейных задач - поиск решения линейных задач симплекс-методом, а для негладких задач - эволюционный поиск решения.

Справка

Найти решение

Закрыть

Sheet: Лист1

Current Sheet

maximise

0

Variable Cells:

Лист1!G13

Лист1!C14:F14

Add

Update

Delete

☒ Unconstrained variables non-negative

Constraints:

<Add new constraint>

Лист1!G18 = Лист1!I18

Лист1!G17 <= Лист1!H7

Лист1!G19 >= Лист1!H9

Selected Constraint:

<=>

Update

Save

Delete

Cancel

Update

Solver: Google Linear Solver

Change

Save Model

Reset Model

☒ Show progress while solving

☒ Check model is linear

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

Минимизировать $Z = 3x_1 + x_2 + 4x_3 + 2x_4$

при условиях:

$2x_1 + x_2 + x_3 \leq 8$

$x_1 + x_3 + x_4 = 5$

$x_2 + x_4 \geq 4$

C	3	1	4	2	10
x	0	0	0	5	
C1	2	1	1	0	0 <= 8
C2	1	0	1	1	5
C3	0	1	0	1	5 >= 4

Реализация кода

```
1. import numpy as np
2.
3.
4. def simplex_method(table, basis, free):
5.     n = table.shape[1] - 1 # количество переменных
6.
7.     # выполняем действия пока не придем к конечному результату
8.     while True:
9.         # в последней строке первые числа - коэффициенты целевой функции
10.        # в последней строке последнее число справа = -Q
11.        # извлекаем последнюю строку без последнего элемента
12.        last_str = table[-1, :-1]
13.        # теперь необходимо найти самый минимальный элемент
14.        j = np.argmin(last_str) # номер разрешающего столбца
15.        # мы не должны выбирать элемент если все элементы положительны
16.        if last_str[j] >= 0:
17.            print('Не можем выбрать столбец, все столбцы положительны')
18.            break
19.        # составим массив, где будет указан элемент из разрешающего столбца, отношение
        bi/aij, причем aij>0
20.        # если aij не будет удовлетворять условию, ставим бесконечность, она не будет
        выбрана
21.        # создаем изначально бесконечный массив
22.        arr = np.full(table.shape[0] - 1, np.inf)
23.        for r in range(table.shape[0] - 1):
24.            if table[r, j] > 0:
25.                arr[r] = table[r, -1] / table[r, j]
26.        if np.all(np.isinf(arr)):
27.            raise ValueError("Задача не ограничена")
28.        # выбираем разрешающую строку
29.        i = np.argmin(arr)
30.        # сохраним разреш элемент
31.        pivot = table[i, j]
32.        old_table = table.copy()
33.
34.        # пересчет ведущей строки
35.        table[i, :] = old_table[i, :] / pivot
36.        # перерасчет разрешающего столбца
37.        table[:, j] = - old_table[:, j] / pivot
38.
39.        # установить значение самого pivot
40.        table[i, j] = 1 / pivot
41.
42.        # пересчет оставшихся элементов
43.        for str_ in range(table.shape[0]):
44.            for col_ in range(table.shape[1]):
45.                # формула пересчета оставшихся элементов
46.                if str_ != i and col_ != j:
47.                    table[str_, col_] = old_table[str_, col_] - (old_table[str_, j] *
                        old_table[i, col_] / pivot)
48.
49.        # замена базиса i <-> j
50.        basis[i], free[j] = free[j], basis[i]
51.
52.        print(table)
53.        print('-----\n')
54.    return table
55.
```

```

56.
57.
58. def to_canonical(c, constraints, nonneg=None, sense='min'):
59.     # приведение к каноническому виду
60.     """
61.
62.     :param c: коэффициенты целевой функции
63.     :param constraints: ограничения
64.     :param nonneg: какие переменные неотрицательны
65.     :param sense: цель минимизация или максимизация
66.     :return:
67.     """
68.     n = len(c)
69.     if nonneg is None:
70.         nonneg = set(range(1, n + 1))
71.
72.     c = [float(v) for v in c]
73.
74.     # 1) max -> min
75.     if sense == 'max':
76.         c = [-v for v in c]
77.     # сколько нужно дополнительных переменных
78.     extra = sum(1 for (_, rel, _) in constraints if rel != '=')
79.     # итого переменных
80.     n_total = n + extra
81.     # необходима для разбитаия переменных на разности  $x_k = (x_k^+) - (x_k^-)$ 
82.     var_map = [('orig', k) for k in range(1, n + 1)]
83.     A, b = [], [] # матрица коэфф, матрица свободных членов
84.     n_extra = n # номер позиции дополнительной переменной
85.     # перебираем ограничения
86.     for coeffs, rel, bi in constraints:
87.         row = [float(v) for v in coeffs] + [0.0] * (n_total - n)
88.         if rel == '<=':
89.             row[n_extra] = 1.0
90.             var_map.append(('slack',)) # новая переменная дополнительная
91.             n_extra += 1
92.         elif rel == '>=':
93.             row[n_extra] = -1.0
94.             var_map.append(('slack',))
95.             n_extra += 1
96.         A.append(row)
97.         b.append(float(bi))
98.     c = c + [0.0] * extra
99.
100.    # 3)  $b_i < 0$  -> строку на -1
101.    for i in range(len(b)):
102.        if b[i] < 0:
103.            A[i] = [-v for v in A[i]]
104.            b[i] = -b[i]
105.
106.    # 4) свободные переменные ->  $x_k = x_k^+ - x_k^-$ 
107.    free_vars = [k for k in range(1, n + 1) if k not in nonneg]
108.    for k in free_vars:
109.        j_k = next(j for j, vm in enumerate(var_map) if vm == ('orig', k))
110.        for i in range(len(A)):
111.            A[i].append(-A[i][j_k])
112.        c.append(-c[j_k])
113.        var_map[j_k] = ('plus', k)
114.        var_map.append(('minus', k))
115.    return np.array(A), np.array(b), np.array(c), var_map
116.

```

```

117.
118. def find_basis_columns(A, m, n):
119.     # пытаемся сразу найти единичный базис (столбец-e_i) для каждой строки
120.     basis_col = [None] * m
121.     used = set()
122.     for i in range(m):
123.         for j in range(n):
124.             if j in used:
125.                 continue
126.             col = A[:, j]
127.             if col[i] == 1 and np.all(np.delete(col, i) == 0):
128.                 basis_col[i] = j
129.                 used.add(j)
130.                 break
131.     return basis_col
132.
133.
134. def solve_general_lp(c, constraints, nonneg=None, sense='min'):
135.     A, b, c_full, var_map = to_canonical(c, constraints, nonneg, sense)
136.     m, n = A.shape
137.
138.     # ищем итоговый единичный базис
139.     basis_col = find_basis_columns(A, m, n)
140.     basis = [None] * m
141.     used = set()
142.     for i in range(m):
143.         if basis_col[i] is not None:
144.             basis[i] = basis_col[i] + 1 # переменные нумеруем с 1
145.             used.add(basis_col[i] + 1)
146.
147.     art_vars = []
148.     next_id = n + 1
149.     for i in range(m):
150.         if basis[i] is None:
151.             art_vars.append(next_id)
152.             basis[i] = next_id
153.             next_id += 1
154.
155.     if art_vars:
156.         free = [k for k in range(1, next_id) if k not in basis]
157.         n_ext = next_id - 1
158.         A_ext = np.hstack([A, np.zeros((m, n_ext - n))])
159.         for i in range(m):
160.             if basis[i] in art_vars:
161.                 A_ext[i, basis[i] - 1] = 1.0
162.
163.         free_cols = [j - 1 for j in free]
164.         table = np.zeros((m + 1, len(free) + 1))
165.         table[:m, :-1] = A_ext[:, free_cols]
166.         table[:m, -1] = b
167.
168.         c_basis = np.array([1.0 if v in art_vars else 0.0 for v in basis])
169.         c_free = np.array([1.0 if v in art_vars else 0.0 for v in free])
170.         table[-1, :-1] = c_free - c_basis @ table[:m, :-1]
171.         table[-1, -1] = -(c_basis @ b)
172.
173.         basis_arr, free_arr = basis[:,], free[:]
174.         table = simplex_method(table, basis_arr, free_arr)
175.         basis, free = basis_arr, free_arr
176.
177.         W1 = -table[-1, -1]

```



```

178.         if abs(W1) > 1e-6:
179.             print(f"W(x*) = {W1:.6f} > 0  =>  X = пусто, решений нет.")
180.             return None
181.
182.         # если искусственная переменная осталась в базисе на уровне 0 -> выводим
eë
183.         for i in range(m):
184.             if basis[i] in art_vars:
185.                 for j, v in enumerate(free):
186.                     if v not in art_vars and abs(table[i, j]) > 1e-9:
187.                         basis_arr, free_arr = basis, free
188.                         pivot = table[i, j]
189.                         old_table = table.copy()
190.                         table[i, :] = old_table[i, :] / pivot
191.                         table[:, j] = -old_table[:, j] / pivot
192.                         table[i, j] = 1 / pivot
193.                         for r in range(table.shape[0]):
194.                             for c_ in range(table.shape[1]):
195.                                 if r != i and c_ != j:
196.                                     table[r, c_] = old_table[r, c_] -
(old_table[r, j] * old_table[i, c_] / pivot)
197.                         basis[i], free[j] = free[j], basis[i]
198.                         break
199.
200.         # убираем столбцы искусственных переменных из free
201.         keep = [j for j, v in enumerate(free) if v not in art_vars]
202.         table = np.hstack([table[:, keep], table[:, -1:]])
203.         free = [free[j] for j in keep]
204.     else:
205.         free = [k for k in range(1, n + 1) if k not in used]
206.         table = np.zeros((m + 1, len(free) + 1))
207.         table[:, :-1] = A[:, [j - 1 for j in free]]
208.         table[:, -1] = b
209.
210.         c_basis = np.array([c_full[v - 1] for v in basis])
211.         c_free = np.array([c_full[v - 1] for v in free])
212.         table[-1, :-1] = c_free - c_basis @ table[:, :-1]
213.         table[-1, -1] = -(c_basis @ table[:, -1])
214.
215.         table = simplex_method(table, basis, free)
216.
217.         # решение
218.         x_full = np.zeros(n)
219.         for i in range(m):
220.             x_full[basis[i] - 1] = table[i, -1]
221.
222.         n_orig = len(c)
223.         x = np.zeros(n_orig)
224.         for j, vm in enumerate(var_map):
225.             if vm[0] == 'orig':
226.                 x[vm[1] - 1] = x_full[j]
227.             elif vm[0] == 'plus':
228.                 x[vm[1] - 1] += x_full[j]
229.             elif vm[0] == 'minus':
230.                 x[vm[1] - 1] -= x_full[j]
231.
232.         W_value = float(np.dot(np.array(c, dtype=float), x))
233.         print('ИТОГ:')
234.         print('x* =', np.round(x, 6))
235.         print('W(x*) =', round(W_value, 6))
236.         return x, W_value

```

```
237.
238.
239.     if __name__ == '__main__':
240.         # тут указываем что хотим сделать (min, max)
241.         target = 'min'
242.         # тут вводим все коэффициенты
243.         # целевая функция
244.         c = [3, 1, 4, 2]
245.         # ограничения
246.         constraints = [
247.             ([2, 1, 1, 0], '<=', 8),
248.             ([1, 0, 1, 1], '=', 5),
249.             ([0, 1, 0, 1], '>=', 4),
250.         ]
251.         solve_general_lp(c, constraints, nonneg={1, 2, 3, 4}, sense='min')
252.         # если не указываете какие переменные неотрицательные то изначально верным
           считается x1....>=0
```

Вывод

В данной лабораторной работе я закрепила на практике навык использования Симплекс-метода. Этот метод необходим для поиска лучшего(оптимального) решения в задачах линейного программирования, когда нужно получить, например. максимум прибыли или минимум затрат при заданных ограничениях.

Задание было выполнено в три больших этапа.

В первом этапе я выполняла ручную реализацию симплекс-метода.

Во втором этапе, я познакомилась с надстройкой Solver и проверила свое ручное решение.

в последнем этапе, я реализовала симплекс-метод с помощью кода.